

Table of Contents

Introduction.....	2
AJAX Http Get method.....	3
Aligning front-end and back-end data.....	7
Run the backend and frontend simultaneously.....	11

Demo: AJAX Http Get

Author: Baifan Zhou

Detailed explanations are created with the help of ChatGPT.

Introduction

In this demo, we shift from using mock data to fetching real data from a backend API for the ItemListPage component.

Key Concepts:

1. **AJAX (Asynchronous JavaScript and XML)** is a technique that allows for asynchronous data retrieval from a server without reloading the entire page.
2. **React Hooks and States:**
 - **useState:** allowing you to track and update dynamic data.
 - **useEffect:** Handles side effects.
3. **Fetch Function:**
 - **fetch:** A modern API for making AJAX HTTP calls.
 - **fetchItems Function:**
 - Sets loading state to true and clears any previous errors.
 - Sends an HTTP GET request to the API endpoint.
 - Parses the JSON response and updates the state with the fetched data.
 - Handles errors and sets loading state to false once the operation completes.
4. **Handling Data Format Issues of Backend and Frontend Alignment:**
 - You can either adjust the backend using Newtonsoft.Json to consistently use PascalCase
 - or adjust the frontend to work with camelCase, which is common in JavaScript and TypeScript.

AJAX Http Get method

AJAX stands for **Asynchronous JavaScript and XML**. It is a technique used to send and receive data from a server asynchronously without needing to reload the entire page. This allows web applications to update parts of the web page dynamically, providing a smoother and more interactive user experience.

Previously, we used mock data for the front-end ItemListPage. This time, we are going to fetch data from the backend API.

We change the ItemListPage.jsx to:

Line1 – Line32

```
ItemListPage.jsx x
myshop > src > items > ItemListPage.jsx > [e] ItemListPage
1  import React, { useState, useEffect } from 'react';
2  import { Table, Button } from 'react-bootstrap';
3
4  const API_URL = 'http://localhost:5043'
5
6  const ItemListPage= () => {
7    const [items, setItems] = useState([]);
8    const [loading, setLoading] = useState(false);
9    const [error, setError] = useState(null);
10
11    const fetchItems = async () => {
12      setLoading(true); // Set loading to true when starting the fetch
13      setError(null); // Clear any previous errors
14
15      try {
16        const response = await fetch(`${API_URL}/api/itemapi/itemlist`);
17        if (!response.ok) {
18          throw new Error('Network response was not ok');
19        }
20        const data = await response.json();
21        setItems(data);
22        console.log(data);
23      } catch (error) {
24        console.error(`There was a problem with the fetch operation: ${error.message}`);
25        setError('Failed to fetch items.');
```

Explanations:

useState: A hook to manage the state within functional components.

useEffect: A hook to handle side effects, like data fetching.

Table and Button from react-bootstrap: Components from the Bootstrap library for styling and layout.

API_URL: A constant that holds the base URL for the API endpoint.

State Management:

- items: Stores the list of items fetched from the API.
- loading: Indicates whether the data is currently being fetched.
- error: Stores any error message that might occur during the fetch operation.

fetchItems Function:

- Sets loading to true to indicate the start of the fetch operation.
- Clears any previous errors by setting error to null.
- Uses the fetch API to send an HTTP GET request to get data from the API endpoint `{API_URL}/api/itemapi/itemlist`.
- The fetch function in JavaScript is a modern API for making HTTP requests. It is used to make AJAX calls and is a replacement for the older XMLHttpRequest API.
- The `response.json()` method is used to parse the JSON data from the response body.
- Line22: prints the data for debugging into the browser console.
- Checks if the response is OK (status in the range 200-299). If not, throws an error.
- Parses the response data as JSON and updates the items state with the fetched data.
- Catches and logs any errors that occur during the fetch process and updates the error state with a user-friendly message.
- Sets loading to false to indicate the end of the fetch operation.

useEffect Hook:

- Calls `fetchItems` when the `ItemPageList` component is loaded (the empty dependency array `[]` ensures it only runs once).
- Handles side effects such as data fetching after the component renders.

In React, hooks and states are fundamental concepts for managing and handling component logic and lifecycle.

Hooks are special functions that let you "hook into" React features such as state and lifecycle methods from functional components. Hooks allow functional components to manage state and side effects, making them more versatile and powerful.

State in React is a way to store and manage dynamic data in a component. It allows a component to keep track of changes and re-render itself when state updates. In class components, state is managed using `this.state` and `this.setState()`. With functional components, `useState` is used to manage state.

useState Hook

The `useState` hook allows you to add state to a functional component. It returns an array with two elements:

1. The current state value.
2. A function to update that state.

Line33 – Line67:

```

33
34   return (
35     <div>
36       <h1>Items</h1>
37       <Button onClick={fetchItems} className="btn btn-primary mb-3" disabled={loading}>
38         {loading ? 'Loading...' : 'Refresh Items'}
39       </Button>
40       {error && <p style={{ color: 'red' }}>{error}</p>}
41       <Table striped bordered hover>
42         <thead>
43           <tr>
44             <th>ID</th>
45             <th>Name</th>
46             <th>Price</th>
47             <th>Descriptions</th>
48             <th>Images</th>
49           </tr>
50         </thead>
51         <tbody>
52           {items.map(item => (
53             <tr key={item.ItemId}>
54               <td>{item.ItemId}</td>
55               <td>{item.Name}</td>
56               <td>{item.Price} NOK</td>
57               <td>{item.Description}</td>
58               <td><img src={` ${API_URL}${item.ImageUrl}`} alt={item.Name} width="120" /></td>
59             </tr>
60           ))}
61         </tbody>
62       </Table>
63     </div>
64   );
65 };
66
67 export default ItemListPage;

```

Line35 – Line63 returns a React component in JSX.

Root Element: <div>: everything needs to be wrapped into one element in the return.

Button:

- **onClick={fetchItems}**: Sets up an event handler that triggers the fetchItems function when the button is clicked.
- **className="btn btn-primary mb-3"**: Applies Bootstrap classes to style the button. btn and btn-primary apply Bootstrap button styles, while mb-3 adds a margin at the bottom.
- **disabled={loading}**: Disables the button while data is being fetched (loading is true).
- **Button Label: {loading ? 'Loading...' : 'Refresh Items'}**: Displays "Loading..." when the loading state is true. Otherwise, it shows "Refresh Items".

Error Message: {error && <p style={{ color: 'red' }}>{error}</p>}: Conditionally renders a paragraph with the error message in red if error is not null or undefined. The && operator is used here for conditional rendering.

****: Displays an image with a source URL constructed from API_URL and item.ImageUrl. Note that the

images come from the server-side, so the **API_URL** is needed before the **item.ImageUrl** to “get” the images from the **wwwroot** folder from the server-side. The alt attribute provides alternative text for accessibility, and the width attribute sets the image width to 120 pixels.

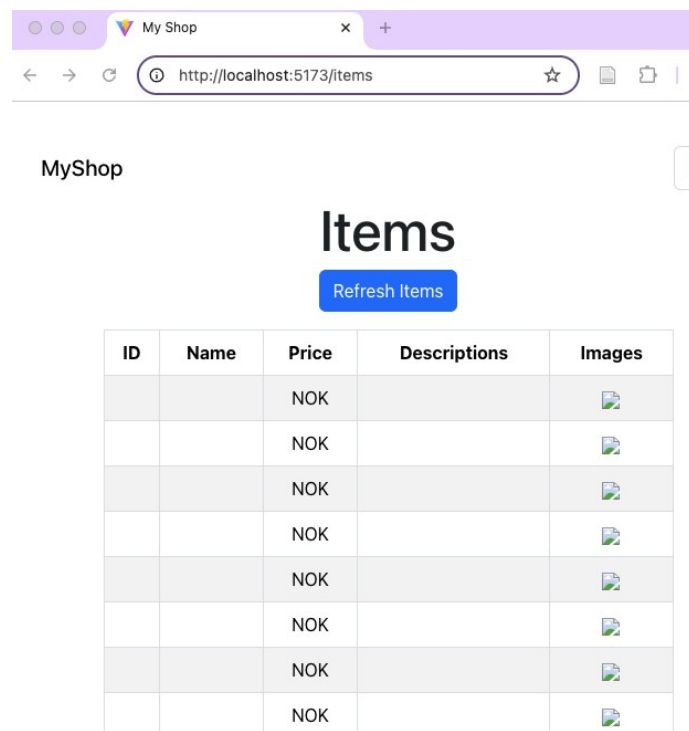
Aligning front-end and back-end data

Now we run the front-end and back-end simultaneously.

Press F5 to run the back-end api.

Go to the terminal, go to the **myshop** folder (`cd myshop`), and run the front-end with `npm run dev` (if your **node_modules** folder is missing, run `npm install` first).

Then we see the webpage that no item data are retrieved, although somehow it knows how many items exist:



Press F12 (right-click, Inspect), go to the Browser Console to see whether the data are retrieved. We can see that the data are indeed retrieved. Then, what's wrong?



If we attentively observe our **ItemTable** in the **ItemListPage.jsx**, we will find that all item have property names with Pascal Case, while in the retrieved data (previous picture), all property names are with camelCase. The reason is the mismatch of property names.

```

51     <tbody>
52         {items.map(item => (
53             <tr key={item.ItemId}>
54                 <td>{item.ItemId}</td>
55                 <td>{item.Name}</td>
56                 <td>{item.Price} NOK</td>
57                 <td>{item.Description}</td>
58                 <td><img src={` ${API_URL}${item.ImageUrl}`} alt={item.Name} width="120" /></td>
59             </tr>
60         ))}
61     </tbody>

```

However, in the backend, our ItemDto.cs actually defines all property names with Pascal Case.

The issue we are encountering is that property names are converted to camelCase in the API responses despite being in Pascal Case in the ItemDto.cs. This is due to the default serialisation behaviour of the JSON serialiser (System.Text.Json) used in the .NET backend.

One way to solve it is to use the NewtonsoftJson serliaser. Change the Line9-12 in api/Program.cs about the NewtonsoftJson to:

```

Program.cs x
api > Program.cs > Program > <top-level-statements-entry-point>
1  using Microsoft.EntityFrameworkCore;
2  using MyShop.DAL;
3  using Serilog;
4  using Serilog.Events;
5  using Newtonsoft.Json.Serialization;
6
7  var builder = WebApplication.CreateBuilder(args);
8
9  builder.Services.AddControllers().AddNewtonsoftJson(options =>
10 {
11     options.SerializerSettings.ReferenceLoopHandling = Newtonsoft.Json.
12     ReferenceLoopHandling.Ignore;
13     options.SerializerSettings.ContractResolver = new DefaultContractResolver();
14 });

```

Now, restart the backend by clicking



Click "Refresh Items", we can see the new data have the correct property names.

MyShop

Items

Refresh Items

ID	Name	Price	Descriptions	Images
1	Pizza	150 NOK	Delicious Italian dish with a thin crust topped with tomato sauce, cheese, and various toppings.	
2	Fried Chicken Leg	20 NOK	Crispy and succulent chicken leg that is deep-fried to perfection, often served as a popular fast food item.	
3	French Fries	50 NOK	Crispy, golden-brown potato slices seasoned with salt and often served as a popular side dish or snack.	
4	Grilled Ribs	250 NOK	Tender and flavorful ribs grilled to perfection, usually served with barbecue sauce.	
5	Tacos	150 NOK	Tortillas filled with various ingredients such as seasoned meat, vegetables, and salsa, folded into a delicious handheld meal.	



The second way to resolve this issue, is to change the naming convention in the ItemTable to camelCase. Which approach you adopt depends on the preference.

If Consistency is Preferred: change the backend: Use Newtonsoft.Json to consistently format all API responses in Pascal Case. This makes your API responses predictable and reduces the need for frontend transformations. This approach is recommended if you want a uniform data format across various clients or if you anticipate multiple types of consumers for your API.

If Flexibility is Preferred: change the frontend: Transform data on the frontend if you need flexibility or if you're working with a variety of data formats. This might be suitable if the backend cannot be changed or if you prefer to handle data transformations closer to where it's used.

We will change the frontend to camelCase since camelCase is the more common convention in JavaScript and TypeScript. We will comment out the line 11 in api/Program.cs that uses Newtonsoft.Json as the serialiser, and change the ItemTable (ItemListPage.jsx) to:

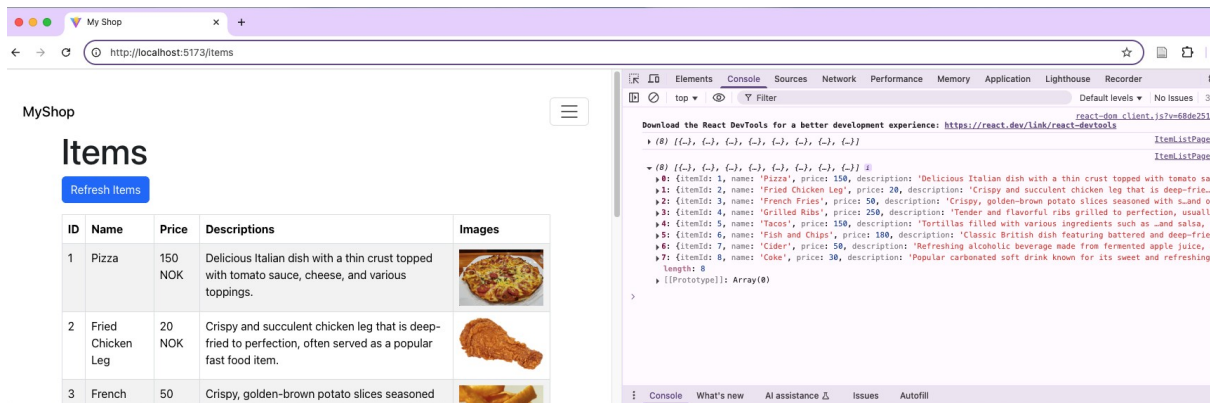
```

51      <tbody>
52        {items.map(item => (
53          <tr key={item.itemId}>
54            <td>{item.itemId}</td>
55            <td>{item.name}</td>
56            <td>{item.price} NOK</td>
57            <td>{item.description}</td>
58            <td><img src={`${API_URL}${item.imageUrl}`} alt={item.name} width="120" /></td>
59          </tr>

```

Restart debugging of the backend (no need to restart the frontend because it reloads automatically each time your frontend code changes).

You will see the exact same webpage but the console shows the retrieved data have a the camelCase property names.



In addition, you can comment out the `text-align: center;` in **App.css** to make the default alignment to be left-aligned.

```

# App.css
myshop > src > # App.css > #root
1  #root {
2    max-width: 1280px;
3    margin: 0 auto;
4    padding: 2rem;
5    /* text-align: center; */
6  }
  
```

Run the backend and frontend simultaneously

We can configure the **package.json** to run both at the same time.

First go to the front-end folder `cd myshop`, then install a tool

```
npm install concurrently --save-dev
```

Modify the **package.json** as below.

```
{} package.json X
myshop > {} package.json > ...
1  {
2    "name": "myshop",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "start-frontend": "vite --open",
8      "start-backend": "dotnet run --project ../api/api.csproj",
9      "dev": "concurrently \"npm run start-backend\" \"npm run start-frontend\"",
10     "build": "vite build",
11     "lint": "eslint .",
12     "preview": "vite preview"
13   },
14   "dependencies": {
15     "bootstrap": "^5.3.7",
16     "react": "^19.1.1",
17     "react-bootstrap": "^2.10.10",
18     "react-dom": "^19.1.1",
19     "react-router-dom": "^7.8.0"
20   },
21   "devDependencies": {
22     "@eslint/js": "^9.33.0",
23     "@types/react": "^19.1.10",
24     "@types/react-dom": "^19.1.7",
25     "@vitejs/plugin-react": "^5.0.0",
26     "eslint": "^9.33.0",
27     "eslint-plugin-react-hooks": "^5.2.0",
28     "eslint-plugin-react-refresh": "^0.4.20",
29     "globals": "^16.3.0",
30     "vite": "^7.1.2"
31   }
32 }
```

Now run the front-end and back-end by running

```
npm run dev
```

You will see both are running.

Note: the backend is running in production mode. To run it in debugging mode, use the method in the previous page.

Congratulations! You now know how to use Http Get from the frontend to retrieve data from the backend API. ☺